

个人开发日志

姓名(组长)：朱子豪 学号：2312190118 班级：计科2301

2025-12-07

- 编写了校园信息聚合论坛系统（校园通）的项目设计文档v1.0
- 编写了校园信息聚合论坛系统（校园通）的API设计文档v1.0
- 编写了校园信息聚合论坛系统（校园通）的数据库设计文档v1.0
- 生成了数据库初始化脚本
- 搭建了后端基础框架（Express + Sequelize + MySQL）
- 搭建了前端基础框架（React + Vite + Ant Design）
- 配置了前后端通信（API代理、跨域处理）
- 实现了项目的目录结构和基础文件

📄 个人开发日志.md	检查项目并更新项目文档	last week
📄 产品需求文档v1.0.md	检查项目并更新项目文档	last week
📄 启动项目指南.md	修改项目后端为python格式	last week
📄 数据库创建与初始化指南.md	修改项目后端为python格式	last week
📄 数据库初始化脚本检查报告.md	修改项目后端为python格式	last week
📄 数据库设计文档v1.0.md	项目基本设计和日志	2 weeks ago
📄 检查报告.md	检查项目并更新项目文档	last week
📄 项目设计文档v1.0.md	检查项目并更新项目文档	last week

2025-12-12

- 编写了《数据库创建与初始化指南》，详细说明了数据库的创建和初始化步骤
- 编写了《启动项目指南》，包含数据库初始化、后端启动和前端启动的完整流程
- 创建了后端项目的package.json文件，添加了Express、Sequelize、MySQL等必要依赖
- 创建了前端项目的package.json文件，添加了React、Ant Design、Axios等必要依赖
- 解决了启动项目时遇到的npm start命令错误问题
- 配置了前后端项目的启动脚本
- 将后端技术栈从Express.js重构为Python Django框架
- 更新了《项目设计文档v1.0》和《API设计文档v1.0》，适配Python后端实现
- 设计并实现了模块化的Django应用结构（accounts、content、likes、reports、ads）
- 实现了自定义User模型和JWT认证系统
- 配置了Django REST Framework用于API开发
- 设置了数据库连接，使用密码3147
- 实现了完整的API endpoints，包括用户认证、内容管理、点赞、举报和广告功能
- 更新了启动项目指南，适配Python后端的启动流程
- 修复了likes应用views.py中的语法错误
- 创建了完整的Python依赖文件requirements.txt
- 删除了旧的Node.js后端文件，清理了项目结构

2025-12-13

今日成果

1. 图标导入错误修复

- 诊断并解决了前端项目中的图标导入错误
- 修复了App.jsx中导入不存在的BriefcaseOutlined图标问题
- 尝试多个替代图标后，最终选择使用已确认存在的FileTextOutlined图标
- 更新了App.jsx中的图标导入语句和使用该图标的地方

2. 项目构建与启动

- 成功运行npx vite build，项目构建无错误
- 启动开发服务器，验证修复效果，服务器成功运行在端口5174上

3. 前端功能完善

- 完善了前端路由配置，添加了完整的页面路由（登录、注册、发帖、帖子详情、编辑帖子、个人资料、管理员后台等）
- 实现了缺失的页面组件：Profile.jsx（个人资料页）、EditPost.jsx（帖子编辑页）
- 完善了PostList组件，实现了帖子列表展示、关键词搜索、板块筛选和多种排序功能
- 提取了板块数据为配置文件（src/config/categories.js），避免硬编码
- 创建了API服务文件（src/services/api.js），封装了axios请求、请求拦截器和响应拦截器
- 添加了状态管理，使用Context API实现了登录状态管理（src/contexts/AuthContext.jsx）
- 修复了CommentList.jsx中缺少UserOutlined图标的问题
- 修复了api.js中的环境变量问题，将process.env替换为Vite支持的import.meta.env
- 创建了.env配置文件，用于配置API地址
- 成功运行构建命令，验证了修复效果

4. 个人中心功能修复

- 修复了个人中心点击无响应的问题
- 实现了编辑资料功能，包括用户名和个人简介修改
- 修复了获取用户信息失败的问题
- 优化了个人信息显示，添加了统计数据

5. 发帖功能修复

- 修复了发布新帖按钮无法点击的问题
- 实现了板块选择功能，用户可以从下拉框中选择板块
- 修复了帖子创建API的500错误
- 在后端数据库中创建了5个预定义板块：校园公告、二手交易、学习交流、休闲娱乐、广告专区

6. API服务优化

- 修复了API URL配置，确保所有请求都使用带斜杠的URL
- 解决了Django APPEND_SLASH机制导致的POST请求500错误
- 优化了API响应拦截器，添加了更详细的错误信息
- 确保前端能正确处理后端返回的数据格式

7. 前端组件修复

- 修复了PostForm组件的板块数据处理逻辑
- 优化了Select组件的实现，确保能正确显示板块列表
- 修复了CreatePost组件的表单提交逻辑
- 优化了Home组件的发布新帖按钮实现

8. 路由配置修复

- 确保所有路由都能正确匹配
- 修复了嵌套路由的配置
- 优化了导航链接的实现

9. 数据库优化

- 在后端数据库中创建了5个板块数据
- 确保用户表结构和数据完整
- 优化了数据库查询性能

10. 后端开发成果

- 修复了后端所有语法错误：
 - 修复了reports/views.py中的Response调用语法错误
 - 修复了reports/serializers.py中的未终止字符串
 - 修复了content/urls.py中的括号问题
 - 修复了reports/apps.py中的字符串问题
- 成功创建并应用了所有数据库迁移：
 - 自定义用户模型已迁移
 - 帖子、评论、分类表已创建
 - 点赞、举报、广告表已创建
- 启动了Django开发服务器：
 - 系统检查通过，无任何错误
 - 服务器运行在 `http://127.0.0.1:8000/`
 - 支持实时文件更改监听
- 所有API端点已按照设计文档实现：
 - 用户认证与管理
 - 内容管理（帖子、评论、分类）
 - 点赞功能
 - 举报功能
 - 广告功能

问题分析与解决

1. 问题1：个人中心点击无响应

- **原因：**路由配置问题
- **解决：**检查并修复了前端路由配置，确保个人中心路由能正确匹配

2. 问题2：获取用户信息失败

- **原因**：API URL配置错误
- **解决**：修复了API服务的URL配置，确保能正确调用后端API

3. 问题3：发布新帖按钮无法点击

- **原因**：按钮组件实现问题
- **解决**：修改了按钮实现，使用useNavigate钩子实现编程式导航

4. 问题4：发帖时无法选择板块

- **原因**：板块数据处理逻辑错误
- **解决**：修复了板块数据处理逻辑，确保能正确提取后端返回的板块数据

5. 问题5：发帖时出现500错误

- **原因**：Django APPEND_SLASH机制导致的POST请求重定向问题
- **解决**：修复了API URL配置，确保所有请求都使用带斜杠的URL

技术要点

前端技术

- React Hooks的使用
- Ant Design组件库的应用
- React Router路由配置
- Axios API服务封装
- 响应式设计

后端技术

- Django REST Framework
- JWT认证机制
- 权限管理
- 数据库模型设计
- API端点配置

数据库技术

- MySQL数据库
- Django ORM
- 数据迁移
- 模型设计

下一步计划

1. 测试所有功能，确保能正常工作
2. 优化用户体验，添加更多交互效果
3. 完善管理后台功能
4. 添加更多测试用例
5. 优化性能，提高系统响应速度

2025-12-14

今日成果

1. 分页实现修复

- 修复了 `accounts/views.py`、`content/views.py`、`ads/views.py` 和 `reports/views.py` 中的分页实现
- 将 `self.paginator.num_pages` 改为 `page.paginator.num_pages`，解决了 `AttributeError`
- 统一使用手动分页逻辑，确保分页数据正确返回

2. 语法错误修复

- 修复了 `content/views.py` 和 `ads/views.py` 中的语法错误
- 解决了括号不匹配问题，确保代码能正常运行

3. 用户举报功能实现

- 在 `PostDetail.jsx` 组件中添加了举报按钮
- 实现了举报模态框，支持选择举报类型和填写举报原因
- 集成了举报提交 API，将举报数据发送到后端
- 支持的举报类型：垃圾广告、色情内容、暴力内容、其他违规

4. 管理员举报管理功能

- 创建了 `ReportManagement.jsx` 组件，用于管理员查看和处理举报
- 在管理员后台添加了举报管理菜单
- 支持按举报类型、状态和关键词搜索举报
- 实现了举报处理功能，支持选择处理动作和添加处理备注
- 支持的处理动作：忽略、删除内容、警告用户

5. API 服务更新

- 在 `api.js` 中添加了管理员举报管理相关的 API 方法
- 完善了前端 API 调用逻辑，确保与后端正确交互

6. 系统API错误修复

- 广告管理API 500错误修复：**
 - 问题：广告管理页面获取广告列表时返回500错误
 - 原因：`AdvertisementUpdateSerializer`类的`validate_content`方法缺少`return value`语句
 - 解决：在`backend/ads/serializers.py`中添加了缺失的`return value`语句
- 板块API 500错误修复：**
 - 问题：板块API的POST请求返回500错误
 - 原因：请求URL缺少尾部斜杠，Django的APPEND_SLASH机制无法在保持POST数据的情况下重定向
 - 解决：在`backend/content/category_urls.py`中为板块详情URL添加了尾部斜杠
- 评论管理数据显示修复：**
 - 问题：评论管理页面显示的数据不完整，缺少所属帖子和评论者信息
 - 原因：前端期望camelCase字段名，但后端返回snake_case字段名

- 解决：在`frontend/src/pages/admin/ContentManagement.jsx`的`fetchComments`函数中添加了数据转换逻辑
- **管理员评论API路由修复：**
 - 问题：管理员评论API返回404错误
 - 原因：缺少专门的管理员评论路由配置
 - 解决：创建了`backend/content/admin_comment_urls.py`，配置了管理员评论列表和删除端点
- **举报管理API 404错误修复：**
 - 问题：举报管理页面获取举报列表时返回404错误
 - 原因：举报API路由配置错误
 - 解决：创建了`backend/reports/admin_report_urls.py`，配置了管理员举报列表和处理端点

7. 评论点赞功能修复

- 修复了评论点赞数量增加两次的问题
- 原因：Like模型的`save/delete`方法和LikeToggleView中都有增减点赞数的逻辑，导致重复执行
- 解决：移除了LikeToggleView中冗余的点赞数增减逻辑，只保留Like模型中的逻辑

8. 管理员后端功能完整实现

- **用户管理**：支持获取用户列表、更新用户角色、封禁/解封用户
- **内容管理**：支持获取帖子列表、删除帖子、置顶/取消置顶帖子
- **评论管理**：新增了获取评论列表、删除评论功能
- **板块管理**：支持获取、创建、更新、删除板块
- **广告管理**：支持获取广告列表、批准/拒绝/删除广告
- **举报管理**：支持获取举报列表、处理举报

9. API端点修复

- 修复了API调用返回404错误的问题
- 修改了`frontend/src/services/api.js`文件中的所有API端点，移除了尾部斜杠
- 确保前端API端点与后端Django路由配置匹配
- 修复了`getCurrentUser`方法的响应处理，确保正确提取用户数据

10. 帖子详情页返回按钮修复

- 修复了帖子详情页返回列表按钮无效的问题
- 将Button与Link组件的结合方式从`Button as={Link}`改为嵌套方式`<Link><Button></Button></Link>`
- 确保所有页面的返回按钮都能正常工作

11. 帖子信息显示修复

- 修复了帖子列表和详情页显示"Invalid Date"的问题
- 将直接访问的用户信息字段改为访问嵌套的`user`对象
- 将驼峰命名的日期字段改为下划线命名，与后端保持一致
- 修复了点赞数量和是否点赞的字段名匹配问题

12. 管理员账号创建

- 创建了一个超级管理员账号，用于管理板块
- 用户名/邮箱：admin@campus.edu.cn
- 密码：admin123
- 角色：superAdmin

13. 管理员登录跳转功能

- 实现了管理员登录后的自动跳转功能
- 管理员用户登录成功后跳转到管理后台
- 普通用户登录成功后跳转到首页
- 在导航栏中添加了管理员入口，只有登录用户可见

14. 删除功能API支持

- 确保了所有删除功能都有对应的后端API支持
- 帖子删除：`postAPI.deletePost` 和 `adminAPI.deletePost`
- 评论删除：`commentAPI.deleteComment` 和 `adminAPI.deleteComment`
- 板块删除：`categoryAPI.deleteCategory` 和 `adminAPI.deleteCategory`
- 广告删除：`adminAPI.deleteAd`

15. 发帖功能简化

- 删除了发帖功能中的帖子类型选项
- 用户发布帖子时只需要选择分区即可
- 后端帖子类型默认为"normal"
- 简化了发帖表单，提高了用户体验

16. 管理员后台修复

- 修复了ContentManagement.jsx中的Tabs组件使用方式
- 将Tabs组件的items属性改为使用TabPane组件作为子元素
- 修复了API 404错误，将管理员帖子和板块管理API端点改为与普通用户相同
- 确保管理员能正常获取帖子列表、评论列表和板块列表
- 修复了API 500错误，确保前端调用正确的后端端点

17. 前端组件修复

- 修复了CategoryManagement组件的导入路径问题
- 更新了AdminDashboard.jsx的注释，确保所有组件都已正确创建
- 确保了所有管理员子页面都能正确导入和使用

18. 管理员后台组件兼容性修复

- 修复了 `CategoryManagement` 组件的类型错误，解决了 "React.jsx: type is invalid" 警告
- 将所有 `Modal` 组件的 `visible` 属性替换为 `open` (antd v5 推荐用法)
- 确保 `UserManagement` 和 `ContentManagement` 组件中的模态框使用正确的属性

19. Tabs 组件用法升级

- 将 `ContentManagement` 组件中的传统 `TabPane` 子组件用法替换为现代 `items` 属性模式
- 移除了过时的 `TabPane` 导入，避免类型错误

- 确保符合 antd v5 的组件使用规范，提高代码可维护性

20. 管理员 API 服务优化

- 调整了前端 `adminAPI` 服务，确保所有管理员请求使用正确的 API 路径
- 修复了广告管理相关的 API 调用路径，确保 `/admin/ads/advertisements` 端点能正确访问
- 更新了板块管理 API，确保创建、更新、删除操作使用正确的管理员路由

21. 后端服务器部署与验证

- 成功启动后端开发服务器，监听地址 `http://0.0.0.0:8000`
- 验证了 API 端点响应正常，特别是 `/api/categories/` 端点返回 200 OK
- 确认后端能正确处理前端请求，返回预期的数据格式

22. 前端与后端协同验证

- 确保前端 API 配置使用正确的 `VITE_API_URL` 环境变量
- 验证了前后端数据交互正常，板块、用户等数据能正确获取
- 修复了可能导致 404 错误的 API 端点配置

23. 权限控制优化

- 在前端 `App.jsx` 中添加了权限检查，只有管理员才能看到管理后台入口
- 修改了 `AuthContext.jsx`，在登录和获取当前用户信息时保存用户角色到 `localStorage`
- 确保普通用户无法直接访问管理员功能

24. 收藏功能实现

- 后端：创建了 `favorites` 应用，实现了收藏/取消收藏、获取收藏列表、检查收藏状态的 API
- 前端：在 `PostDetail.jsx` 组件中添加了收藏按钮，支持收藏/取消收藏帖子
- 使用 `favoriteAPI` 调用后端 API，实现收藏状态的实时更新

25. 关注功能实现

- 后端：创建了 `follows` 应用，实现了关注/取消关注、获取关注列表、检查关注状态的 API
- 前端：在 `PostDetail.jsx` 组件中添加了关注按钮，支持关注/取消关注帖子作者
- 实现了关注状态的实时更新和提示

26. 帖子上传视频功能

- 后端：修改了 `Post` 模型，将 `images` 字段改为 `media_files`，支持图片和视频 URL 数组
- 前端：修改了 `PostForm.jsx` 组件，扩展上传功能以支持视频文件
- 修改了 `PostDetail.jsx` 组件，根据文件类型（图片/视频）使用不同的渲染方式
- 配置了媒体文件的上传和展示逻辑

27. 修复 Profile.jsx 中的头像上传问题

- 修复了 `Profile.jsx` 中的 `avatarFile` 状态更新问题
- 添加了 `onChange` 事件处理程序，确保头像文件被正确保存到状态中
- 确保用户可以成功上传和更新头像

28. 浏览历史功能与 bug 修复

- **实现用户浏览历史功能：**
 - 在 `backend/content/views.py` 中更新了 `retrieve` 方法，实现了浏览历史记录功能
 - 使用 `timezone.now()` 确保准确记录用户浏览时间
 - 实现了 `BrowsingHistoryListAPIView` 和 `BrowsingHistoryClearAPIView` 视图
 - 确保 API 端点返回一致的响应格式
- **修复浏览历史记录问题：**
 - 修复了浏览历史记录时间不准确的问题，将 `post.updated_at` 改为 `timezone.now()`
- **统一 API 响应格式：**
 - 更新了所有浏览历史 API 视图，确保返回标准化的响应对象
- **实现前端浏览历史功能：**
 - 在 `Profile.jsx` 组件中添加了浏览历史标签页
 - 实现了浏览历史列表展示，支持分页和排序
 - 添加了清除历史功能，支持用户一键清空浏览记录
 - 集成了后端 API，实现数据的获取和清除
- **修复个人资料更新 400 错误：**
 - 修复了 `Profile.jsx` 中缺少状态变量 `avatarFile` 和 `avatarPreview` 的问题
- **添加浏览历史 API 服务：**
 - 在 `api.js` 中添加了 `historyAPI` 对象，包含获取和清除浏览历史的方法
- **修复关注功能失败问题：**
 - 分析了关注功能失败的原因：前端 API 调用路径缺少尾部斜杠
 - 修改了 `api.js` 中的 API 调用路径，为所有相关 API 添加了尾部斜杠
 - 修复的 API 包括：`followAPI.toggleFollow`、`followAPI.checkFollow`、`favoriteAPI.toggleFavorite`、`favoriteAPI.checkFavorite`、`favoriteAPI.getFavorites`

问题分析与解决

1. 评论点赞数量增加两次问题

- **原因：**点赞功能的逻辑被实现了两次，Like模型和LikeToggleView中都有增减点赞数的代码
- **解决：**删除了LikeToggleView中的冗余逻辑，只保留Like模型中的逻辑

2. 管理员后端功能不完整问题

- **原因：**前端adminAPI中定义了多个管理功能，但后端缺少对应的实现
- **解决：**为后端添加了帖子置顶/取消置顶、评论管理等功能，确保所有前端API调用都有后端支持

3. API调用返回404错误问题

- **原因：**前端API端点URL带有尾部斜杠，与后端Django路由不匹配
- **解决：**移除了所有API端点URL中的尾部斜杠

4. 帖子详情页返回按钮无效问题

- **原因：**Ant Design v5不再支持Button组件的as属性
- **解决：**将Button与Link组件的结合方式从 `Button as={Link}` 改为嵌套方式

5. 帖子信息显示"Invalid Date"问题

- **原因：**前端直接访问了不存在的user属性，且日期字段名与后端不匹配

- **解决**：改为访问嵌套的user对象，将驼峰命名改为下划线命名

6. CategoryManagement组件类型无效问题

- **原因**：可能是浏览器缓存或热重载问题导致的
- **解决**：更新了AdminDashboard.jsx的注释，确保所有组件都已正确导入

7. 管理员登录后未跳转问题

- **原因**：登录成功后没有根据用户角色进行不同的跳转
- **解决**：添加了根据用户角色的跳转逻辑

8. 管理员后台出现组件类型错误问题

- **原因**：Tabs组件使用了错误的items属性语法
- **解决**：改为使用TabPane组件作为子元素

9. CategoryManagement组件类型错误问题

- **原因**：使用了过时的 `TabPane` 组件，而 `antd v5` 中 `Tabs` 组件已改为使用 `items` 属性
- **解决**：将 `Tabs` 组件重构为使用 `items` 属性，移除 `TabPane` 导入

10. Modal组件警告问题

- **原因**：使用了已废弃的 `visible` 属性，`antd v5` 推荐使用 `open` 属性
- **解决**：将所有模态框的 `visible` 属性替换为 `open`

11. 管理员API端点配置问题

- **原因**：前端API路径与后端路由不完全匹配
- **解决**：调整前端 `adminAPI` 服务中的路径，确保与后端 `urls.py` 配置一致

技术要点

- **前后端分离架构**：前端使用 React + Ant Design，后端使用 Django + Django REST Framework
- **JWT 认证**：确保 API 调用的安全性
- **RESTful API 设计**：遵循 REST 原则，API 命名规范清晰
- **响应式设计**：适配不同屏幕尺寸
- **组件化开发**：提高代码复用性和可维护性
- **前后端数据格式统一**：确保前端和后端使用一致的数据字段命名规范
- **API路由设计**：合理配置API路由，区分管理员和普通用户权限
- **序列化器验证**：确保序列化器的验证方法完整，包括必要的返回语句
- **错误日志分析**：通过分析浏览器控制台和后端服务器日志，准确定位问题根源
- **数据转换处理**：在前端或后端进行必要的数据格式转换，确保数据能正确解析和显示
- **权限管理**：使用JWT令牌和角色检查，实现精细的权限控制
- **前后端协同开发**：确保前后端API调用和数据格式一致
- **媒体文件处理**：支持多种媒体类型，根据文件类型动态渲染
- **状态管理**：使用React状态管理组件状态，确保UI与数据同步
- **Django ORM**：使用 `update_or_create` 方法高效管理浏览历史记录
- **React Hooks**：使用 `useState` 和 `useEffect` 管理组件状态和副作用
- **Ant Design组件库**：使用 `Card`、`Spin`、`List`、`Avatar`、`Button` 等组件构建用户界面

- **RESTful API调用**：调用后端提供的关注相关 API，实现数据交互
- **时间戳管理**：使用 `django.utils.timezone` 确保时间记录准确
- **Django APPEND_SLASH机制**：理解并解决了 POST 请求重定向导致的数据丢失问题
- **前端API调用最佳实践**：确保 API 调用路径与后端路由配置一致

2025-12-15

今日成果

1. DeepSeek API 集成

- 将 DeepSeek API 集成到 AI 助手应用中，替换了原来的模拟 AI 生成逻辑
- 更新了 `_generate_ai_response`、`generate_post_title`、`generate_post_summary`、`expand_post_content` 和 `enhanced_search` 方法
- 添加了通用的 `_call_deepseek_api` 方法，封装了 DeepSeek API 调用逻辑
- 为不同的 AI 功能设置了不同的温度参数和最大令牌数，优化生成效果
- 添加了备用方案，以防 API 调用失败，确保系统可靠性

2. 路由修复

- 修复了前端和后端的路由，确保所有路由都使用了正确的尾随斜杠
- 修复了以下文件的路由：
 - `backend/accounts/urls.py`：认证、用户和管理员相关路由
 - `backend/content/post_urls.py`：帖子相关路由
 - `backend/content/comment_urls.py`：评论相关路由
 - `backend/content/admin_comment_urls.py`：管理员评论相关路由
 - `backend/likes/urls.py`：点赞相关路由
 - `backend/reports/urls.py`：举报相关路由
 - `backend/reports/admin_report_urls.py`：管理员举报相关路由
 - `frontend/src/services/api.js`：前端 API 调用路径
- 确保所有 API 调用都与 Django 的 APPEND_SLASH 机制兼容

3. 测试验证

- 重新启动后端开发服务器，监听地址 `http://0.0.0.0:8000`
- 使用 PowerShell 测试 API 端点，确认 AI 相关端点返回 200 OK 响应
- 验证了修复后的路由能正确处理请求，不再返回 404 错误
- 确保前后端数据交互正常，API 调用成功

4. AI 内容审核功能优化

- 增强了敏感词过滤机制，添加了更多中文敏感词
- 优化了敏感词替换算法，确保长词优先匹配
- 修复了敏感词过滤中的逻辑错误，确保所有敏感词都能被正确检测和替换
- 添加了敏感词检测的测试用例，确保功能可靠性

5. 前端组件优化

- 优化了 PostList 组件的性能，减少不必要的渲染
- 改进了 PostDetail 组件的布局，提高用户体验

- 修复了 CommentList 组件中的样式问题
- 增强了表单验证，添加了更友好的错误提示

6. 数据库优化

- 为频繁查询的字段添加了索引，提高查询性能
- 优化了数据库表结构，减少冗余数据
- 清理了过期数据，提高数据库效率

7. 部署准备

- 检查并更新了 requirements.txt，确保所有依赖都已正确配置
- 优化了项目结构，准备部署到生产环境
- 编写了部署文档，详细说明了部署步骤和注意事项

8. 关注列表功能实现

- **个人中心关注列表功能实现：**
 - 在 `Profile.jsx` 组件中添加了关注列表和粉丝列表功能
 - 实现了两个新的标签页："我的关注"和"我的粉丝"
 - 集成了 `followAPI` 调用后端 API，实现关注数据的获取和渲染
 - 支持关注/取消关注操作，实现了关注状态的实时更新
 - 添加了加载状态显示，提高用户体验
- **关注列表数据获取和渲染：**
 - 添加了 `following` 状态变量，用于存储关注列表数据
 - 实现了 `fetchFollowing` 函数，用于获取当前用户的关注列表
 - 处理了不同的响应格式，确保数据能正确解析和显示
 - 渲染关注列表，显示用户头像、用户名、个人简介、发帖数和获赞数
- **粉丝列表数据获取和渲染：**
 - 添加了 `followers` 状态变量，用于存储粉丝列表数据
 - 实现了 `fetchFollowers` 函数，用于获取当前用户的粉丝列表
 - 渲染粉丝列表，显示用户头像、用户名、个人简介、发帖数和获赞数
- **关注/取消关注功能实现：**
 - 实现了 `handleToggleFollow` 函数，用于处理关注/取消关注操作
 - 调用 `followAPI.toggleFollow` 接口，实现关注状态的切换
 - 更新关注列表和粉丝列表，确保数据的实时性
- **标签页切换数据加载：**
 - 在 `useEffect` 钩子中添加了标签页切换监听

- 实现了按需加载，提高性能



9. AI功能实现

○ AI智能问答机器人实现：

- 后端：创建了`ai_assistant`应用，实现了校园知识问答机器人的核心功能
- 实现了`KnowledgeBase`模型，用于存储常见问题和答案
- 实现了`AIResponseLog`模型，用于记录AI响应日志
- 开发了`AIAssistantService`服务类，包含文本预处理、相似度计算、知识问答、帖子创作辅助和智能搜索等核心功能
- 实现了`sklearn`优雅降级方案，当`sklearn`库不可用时自动切换到基于Jaccard相似度的简单实现

○ AI API端点实现：

- 创建了完整的AI助手API端点：
 - `/api/ai/ask`：智能问答
 - `/api/ai/generate-title`：生成帖子标题
 - `/api/ai/generate-summary`：生成帖子摘要
 - `/api/ai/expand-content`：根据关键词扩展内容
 - `/api/ai/enhanced-search`：智能搜索增强
- 配置了`KnowledgeBaseViewSet`和`AIResponseLogViewSet`用于知识库管理
- 将AI助手路由添加到主项目的URL配置中

○ 前端AI功能集成：

- 创建了`AIAssistant.jsx`组件，实现了校园知识问答机器人界面
- 在`PostForm.jsx`组件中添加了AI辅助功能：AI生成标题、AI生成摘要、AI扩展内容
- 在`PostList.jsx`组件中添加了智能搜索增强功能，支持自然语言搜索和搜索建议
- 更新了`Home.jsx`，将AI助手组件集成到首页
- 在`api.js`中添加了`aiAPI`，用于调用AI相关的后端API

○ 数据库迁移：

- 运行了`python manage.py makemigrations`，创建了AI助手应用的数据库迁移文件
- 运行了`python manage.py migrate`，实际创建了AI助手应用所需的数据库表
- 解决了MySQL CharField最大长度限制警告

○ AI功能依赖处理：

- 尝试安装`scikit-learn`和`jieba`依赖库
- 实现了`sklearn`库的优雅降级方案，确保在缺少该库时系统仍能正常运行

- 配置了文本预处理和相似度计算的备选方案

10. 收藏夹功能完善

- 个人中心收藏夹功能实现：
 - 在`Profile.jsx`组件中添加了"我的收藏"标签页
 - 实现了收藏列表的展示，包含帖子标题、内容摘要、点赞数、评论数和收藏时间
 - 添加了`favorites`状态变量，用于存储用户的收藏列表数据
 - 实现了`fetchFavorites`函数，用于从后端获取收藏列表
 - 添加了`favoritesLoading`状态，用于显示加载状态
- 帖子列表收藏功能：
 - 在`PostList.jsx`组件中添加了收藏按钮
 - 实现了`favoritedPosts`状态，用于管理每个帖子的收藏状态
 - 实现了`favoritingPosts`状态，用于管理正在收藏/取消收藏的帖子
 - 实现了`checkFavoriteStatus`函数，用于检查帖子的收藏状态
 - 实现了`handleToggleFavorite`函数，用于处理收藏/取消收藏操作
- 帖子详情页收藏功能增强：
 - 确认`PostDetail.jsx`中已有的收藏功能正常工作
 - 确保收藏状态能实时更新
 - 优化了收藏按钮的样式和交互体验
- API调用优化：
 - 确保所有收藏相关的API调用都使用正确的URL格式
 - 验证了`favoriteAPI`的`toggleFavorite`、`checkFavorite`和`getFavorites`方法能正确调用后端API

技术要点

1. AI模型集成

- 实现了基于知识库的问答系统，结合文本相似度计算
- 实现了`sklearn`库的优雅降级方案，确保在缺少该库时系统仍能正常运行
- 使用`jieba`进行中文分词，实现了文本预处理和相似度计算

2. Python依赖管理

- 处理了第三方库的安装和依赖问题，实现了优雅降级
- 配置了文本预处理和相似度计算的备选方案

3. 前后端协同开发

- 前端组件与后端API无缝集成，实现了完整的AI功能
- 设计了知识库和AI响应日志的数据库模型
- 创建了符合REST原则的AI相关API端点

4. React组件开发

- 使用React和Ant Design构建了用户友好的AI功能界面
- 使用React Hooks管理组件状态和副作用
- 使用Ant Design组件库构建用户界面

5. DeepSeek API调用

- 使用requests库调用外部AI API，实现高质量内容生成
- 配置了不同AI功能的温度参数和最大令牌数，优化生成效果
- 添加了备用方案，处理API调用失败情况

6. 路由管理

- 理解并解决了Django APPEND_SLASH机制导致的POST请求重定向问题
- 确保前端API调用路径与后端路由配置一致

7. 敏感词过滤算法

- 使用Trie树结构实现高效的敏感词检测和替换
- 优化了敏感词替换算法，确保长词优先匹配

8. 前端性能优化

- 使用React.memo和useMemo减少不必要的渲染
- 改进了组件布局，提高用户体验

9. 数据库性能优化

- 添加索引和优化查询语句
- 优化数据库表结构，减少冗余数据

10. 部署准备

- 确保项目能顺利部署到生产环境
- 编写了部署文档，详细说明了部署步骤和注意事项

2025-12-16

今日成果

1. 主界面UI优化

- 重新排序了首页组件，将帖子和搜索功能放在AI问答上方
- 为Home.jsx添加了现代化的渐变横幅和卡片设计
- 增强了视觉效果，添加了阴影、圆角和悬停效果
- 优化了组件间距，提升了页面整体美观度

2. 帖子管理修复

- 修复了帖子管理中状态栏不会因编辑而改变的问题
- 为PostListSerializer添加了status字段，确保编辑后状态正确更新
- 实现了完整的帖子状态管理，包括发布、草稿、审核中等状态

3. 帖子详情页私信功能实现

- 在PostDetail.jsx组件中添加了点击发帖人头像查看信息的功能
- 实现了用户资料模态框，显示发帖人详细信息
- 添加了私信功能，支持直接与发帖人进行私密交流

- 实现了私信抽屉组件，提供友好的聊天界面
- 支持关注/取消关注发帖人功能

4. 私信发送功能修复

- 修复了私信发送时返回500错误的问题
- 解决了MessageCreateSerializer中的recipient_id字段配置错误
- 实现了正确的create方法来处理私信创建
- 确保序列化器能正确获取请求上下文
- 修复后私信可以正常发送

5. Settings页面修复

- 修复了useNavigate的错误使用方式，将const [navigate] = useNavigate()改为const navigate = useNavigate()
- Settings页面现在可以正常访问和使用
- 开发了完整的设置中心，包含账户、密码、隐私、通知等多个标签页
- 实现了真实API调用的密码修改功能
- 完善了隐私设置表单，支持多种隐私选项的切换

6. 夜间模式实现

- 添加了完整的深色主题CSS样式
- 调整了ConfigProvider的包裹顺序，确保主题配置正确应用
- 主题切换开关可以正常工作，实现明暗主题切换
- 添加了平滑的过渡效果，提升用户体验

7. 隐私设置功能实现

- 后端：为User模型添加了5个隐私设置字段：
 - show_profile：控制谁可以查看个人资料
 - allow_messages：控制谁可以发送私信
 - allow_follow：允许他人关注
 - show_posts：控制谁可以查看帖子
 - save_history：是否保存浏览历史
- 更新了用户序列化器，添加了隐私设置字段
- 运行了数据库迁移，将新字段应用到数据库
- 前端：实现了隐私设置表单，支持保存和加载隐私设置 G:\web_Final-project\image\已使用\隐私设置.png

8. API服务完善

- 在api.js中为authAPI添加了changePassword方法
- 修复了API端点路径，确保一致的尾部斜杠处理
- 修复了API服务中重复的/api前缀问题
- 确保所有API调用都使用真实API，不再使用前端模拟数据

9. PostList组件优化

- 改进了帖子列表的样式设计，使用更好的卡片布局
- 添加了空状态处理，显示友好的提示信息

- 增强了搜索和筛选UI，提升用户体验

10. Feedback组件开发

- 创建了Feedback.jsx组件，用于收集用户反馈
- 实现了反馈表单和提交功能
- 支持多种反馈类型的选择

11. 前端错误修复

- 修复了Spin组件的tip属性使用问题，改为使用自定义布局
- 修复了图标导入错误，将不存在的ShieldOutlined替换为KeyOutlined
- 成功运行前端构建，确保代码没有语法错误

12. 前后端服务优化

- 后端服务运行稳定，API端点正常响应
- 前端服务运行稳定，页面加载正常
- 所有API调用都能正确处理，无404或500错误

问题分析与解决

1. 帖子管理状态栏不更新

- 原因：PostListSerializer中缺少status字段，导致编辑后状态无法正确返回
- 解决：为序列化器添加status字段，确保状态信息正确传递到前端

2. authAPI.changePassword is not a function错误

- 原因：api.js中authAPI对象缺少changePassword方法
- 解决：在authAPI中添加changePassword方法，并实现对应的后端视图

3. 私信发送500错误

- 原因：MessageCreateSerializer中的recipient_id字段配置错误，导致无法正确处理收件人ID
- 解决：移除了错误的source='recipient'配置，实现了正确的create方法来处理私信创建

4. Settings页面无法访问

- 原因：useNavigate的错误使用方式，将其作为数组解构
- 解决：将const [navigate] = useNavigate()改为const navigate = useNavigate()

5. 夜间模式不生效

- 原因：缺少深色主题的CSS样式定义
- 解决：添加了完整的深色主题CSS样式，调整了ConfigProvider的包裹顺序

6. 隐私设置无法保存

- 原因：缺少后端支持和前端保存逻辑
- 解决：为User模型添加了隐私设置字段，实现了前端保存逻辑

7. 前端构建失败

- **原因**：图标导入错误和其他语法问题
- **解决**：修复了图标导入错误，修复了其他语法问题

8. API 404错误

- **原因**：前后端API端点路径不匹配，缺少或多了尾部斜杠
- **解决**：统一了所有API端点的尾部斜杠处理，确保前后端一致

技术要点

1. 后端技术

- Django模型扩展：为现有模型添加新字段
- 数据库迁移：创建和应用数据库迁移文件
- 序列化器定制：实现自定义的create方法处理复杂数据
- API设计：确保API端点的正确性和可靠性
- Django REST Framework：实现完整的RESTful API
- JWT认证：确保API调用的安全性

2. 前端技术

- React Hooks：正确使用useNavigate、useState、useEffect等钩子
- Ant Design组件库：使用Modal、Drawer、Card等组件构建界面
- React Router：实现页面路由和导航
- Ant Design主题：使用ConfigProvider实现主题切换
- CSS样式：编写响应式和主题化的CSS
- 表单处理：实现复杂表单的获取和保存逻辑
- 组件化开发：创建可复用的UI组件

3. 前后端协同

- API设计：确保前后端API调用一致
- 数据格式：确保前后端数据格式匹配
- 错误处理：实现统一的错误处理机制
- RESTful API设计：遵循REST原则设计API端点

4. 问题诊断与调试

- 使用浏览器开发者工具调试前端错误
- 查看后端日志定位服务器错误
- 使用Postman或PowerShell测试API端点
- 通过代码分析定位问题根源

后续开发成果（2025-12-16 下午）

1. 帖子管理页面错误修复

- 修复了ContentManagement.jsx中筛选函数的空值检查问题
- 解决了toLowerCase()报错，确保在调用toLowerCase()前检查值是否存在

- 修复了帖子筛选逻辑，确保只有存在的字段才会被用于筛选

2. 帖子编辑功能实现

- 在adminAPI中添加了updatePost方法，用于更新帖子信息
- 实现了编辑帖子的模态框，包含标题、内容和状态编辑
- 支持更新帖子信息并实时刷新列表
- 实现了完整的帖子状态管理，包括正常和隐藏两个状态

3. 用户管理操作栏UI优化

- 将操作按钮改为圆形图标按钮，减少空间占用
- 使用Tooltip组件添加操作提示
- 增加了操作栏宽度，确保所有按钮正常显示
- 调整了按钮间距，提升视觉效果

4. 帖子状态选项简化

- 将编辑帖子的状态选项从5个简化为只有2个：正常和隐藏
- 在帖子列表中添加了对"隐藏"状态的标签显示
- 实现了帖子状态的完整管理和展示

5. 设置中心功能添加

- 创建设置中心页面(Settings.jsx)，包含4个标签页：账号管理、密码管理、隐私管理和通知设置
- 在App.jsx中添加了设置页面的路由配置
- 在导航栏中添加了设置中心的入口
- 在个人中心页面添加了"账号设置"标签，链接到设置中心
- 支持用户通过多种方式访问设置中心

设置中心

账号管理

密码管理

隐私管理

* 用户名

zzh

* 邮箱

test@campus.edu.cn

个人简介

231

保存

取消

退出登录

反馈

6. PostDetail组件错误修复

- 修复了PostDetail.jsx中useNavigate的错误使用方式，将`const [navigate] = useNavigate()`改为`const navigate = useNavigate()`
- 添加了缺失的SendOutlined图标导入
- 确保组件能正常渲染和使用

7. 组件弃用属性修复

- 修复了PostList.jsx中的bodyStyle弃用属性，将其替换为`styles={{ body: { padding: '16px 24px 20px' } }}`
- 确保组件符合Ant Design v5的最佳实践
- 消除了浏览器控制台的警告信息

技术要点

- **React组件开发**：正确使用React Hooks，特别是useNavigate钩子
- **Ant Design组件库**：使用最新版本的组件属性和最佳实践
- **API服务设计**：设计和实现完整的CRUD操作
- **UI/UX优化**：优化操作栏设计，提升用户体验
- **错误处理**：添加适当的空值检查，防止运行时错误
- **前后端协同**：确保前端API调用与后端实现一致
- **代码质量**：遵循React和Ant Design的最佳实践，编写可维护的代码

2025-12-18

今日成果

1. 评论区头像点击功能实现

- 在CommentList.jsx组件中为评论用户头像添加了点击事件
- 实现了用户信息模态框，显示评论者的详细信息
- 添加了关注/取消关注功能，支持直接在模态框中关注或取消关注评论者
- 实现了私聊功能，支持跳转到私信页面与评论者进行私密交流
- 模态框中显示用户的基本信息、个人简介和统计数据（发帖数、获赞数）

2. 组件功能增强

- 导入了userAPI和followAPI，实现了与后端的交互
- 添加了加载状态和错误处理，提升用户体验
- 使用Ant Design组件库实现了美观的用户界面
- 实现了关注状态的实时更新和提示

3. 技术实现

- 使用React Hooks管理组件状态
- 调用后端API获取用户信息和关注状态
- 实现了关注/取消关注的API调用
- 实现了模态框的显示和隐藏逻辑
- 优化了组件的性能和用户体验

技术要点

1. 前端技术

- React Hooks : useState、useEffect等
- Ant Design组件库 : Modal、Card、Button、Avatar等
- API调用 : 使用封装好的API服务调用后端接口
- 组件化开发 : 实现可复用的用户信息模态框

2. 后端技术

- RESTful API设计 : 遵循REST原则设计API端点
- JWT认证 : 确保API调用的安全性
- 数据模型 : User、Follow等模型的应用
- 序列化器 : 处理前后端数据交互

3. 用户体验优化

- 加载状态显示 : 使用Spin组件显示加载状态
- 错误处理 : 使用message组件显示错误提示
- 交互反馈 : 操作成功或失败时显示相应提示
- 视觉设计 : 使用Ant Design的设计规范, 确保界面美观易用

2025-12-19

今日成果

1. AI帖子创作辅助功能优化

- 优化了PostForm.jsx中的关键词提取逻辑, 从标题和内容中智能提取关键词
- 使用多种分隔符 (空格、逗号、句号等) 分割文本, 提高关键词提取准确性
- 添加了关键词去重和数量限制, 确保生成内容的相关性
- 改进了前端请求逻辑, 添加了更详细的错误处理
- 优化了AI提示词, 确保生成内容符合要求的字数和风格
- 修复了前端URL扩展名判断视频类型失效的问题, 改为根据文件MIME类型判断

章铁飞老师课讲的真好，视频录的也不错

内容扩展成功

Q AI 帖子创作辅助

AI生成标题AI生成摘要AI扩展内容

* 所属板块

请选择板块

* 帖子内容

章铁飞老师课讲的真好，视频录的也不错

章铁飞老师的课，让我重新爱上了学习！

最近在校论坛上，经常看到同学们讨论章铁飞老师的课程，我也忍不住想分享一下自己的感受。作为一名曾经对某些学科提不起兴趣的学生，章铁飞老师的课堂真的让我眼前一亮！

“首先，必须夸夸章铁飞老师课讲的真好！”他的授课风格特别生动，总是能把复杂的知识点拆解成易懂的小故事或例子。记得有一次他讲解一个抽象的理论，居然用日常生活中的场景来比喻，瞬间就让我豁然开朗。而且，他非常注重互动，经常鼓励我们提问，课堂氛围轻松又积极，完全不会感到枯燥或压力。

“更值得一提的是，他视频录的也不错！”作为补充学习材料，这些视频清晰度、内容精炼，我经常在课后反复观看，巩固重点。尤其是考前复习阶段，这些视频简直就是“救命神器”，帮我快速回顾关键概念，效率大大提升。无论是节奏把握还是知识点梳理，都能看出老师的用心。

如果你还在为某个科目头疼，或者想找点高质量的学习资源，我真的强烈推荐关注章铁飞老师的课程。他的课堂不仅传授知识，更激发了我们的学习热情。相信你会像我一样，从中收获满满的惊喜和成长！

大家有没有上过章老师的课呢？一起来聊聊你的体验吧！

2. 视频上传功能修复

- 修复了帖子创建时视频上传失败的问题

- 修复了前端文件上传axios实例配置，移除了显式设置的Content-Type，让浏览器自动设置
- 在axios请求拦截器中添加了FormData类型检测，移除默认的Content-Type
- 优化了视频文件的预览逻辑，根据文件MIME类型而非URL扩展名判断文件类型
- 确保后端FileUploadView支持视频文件上传，包括MP4、MOV、AVI等格式
- 优化了文件上传状态管理，提供清晰的上传进度和结果反馈
- 修复了预览功能，确保视频文件能够正确预览

3. 个人中心帖子删除功能

- 在Profile.jsx组件中添加了删除帖子相关的状态管理
- 实现了删除确认对话框，防止用户误操作
- 调用postAPI.deletePost方法执行删除操作
- 删除成功后实时更新帖子列表，提供良好的交互反馈
- 添加了加载状态和错误处理，提升用户体验
- 为每个帖子添加了删除按钮，显示在操作区

4. 私信中心功能修复

- 修复了私信中心一键标为已读功能，将is_unread字段从BooleanField改为SerializerMethodField
- 修复了同一用户聊天分出两个聊天的问题，优化了对话用户ID获取逻辑
- 修复了从对话详情发送私信时缺少recipient_id字段的问题

5. 通知功能修复

- 修复了通知功能中的is_unread字段定义，改为使用SerializerMethodField
- 实现了正确的get_is_unread方法，确保通知状态正确显示

6. 帖子点赞和评论统计修复

- 创建了content/signals.py文件，实现了实时更新帖子点赞数和评论数的信号处理函数
- 修复了无法对帖子点赞的问题，添加了点赞相关状态管理和事件处理
- 修复了帖子评论计数功能无效的问题
- 修复了点赞一次出现两个点赞的问题，移除了Like模型中手动的点赞数更新逻辑
- 创建并运行了update_post_stats管理命令，手动更新所有帖子的统计信息

问题分析与解决

1. AI扩展内容相关性不高

- **原因：**关键词提取不准确，导致生成内容相关性不高
- **解决：**优化关键词提取逻辑，使用多种分隔符分割文本，添加关键词去重和数量限制

2. 视频上传失败

- **原因：**前端显式设置Content-Type导致视频上传失败
- **解决：**移除axios实例中显式设置的Content-Type，让浏览器自动设置带有边界字符串的multipart/form-data

3. 视频预览功能失效

- **原因：**根据URL扩展名判断视频类型失效
- **解决：**保存文件MIME类型，根据类型渲染不同的预览组件（video或img）

4. 缺少删除确认机制

- **原因**：个人中心帖子删除功能没有确认机制，容易导致误操作
- **解决**：添加删除确认对话框，使用Modal组件实现，让用户确认后再执行删除操作

5. 私信图片加载问题

- **原因**：前端错误地使用API URL作为媒体文件的基础URL，导致图片无法加载
- **解决**：将API URL中的/api后缀替换掉，生成正确的媒体文件访问URL

6. 私信消息显示不友好

- **原因**：所有消息都显示在同一侧，没有区分发送者
- **解决**：根据消息的sender.id与当前用户id比较，确定消息显示位置，我的消息显示在右侧，别人的消息显示在左侧

7. 前端文件上传UI不显示

- **原因**：文件列表管理不当，Upload组件配置错误
- **解决**：手动管理媒体文件列表，使用状态存储上传文件，修复Upload组件配置

8. 文件上传路由配置问题

- **原因**：Django APPEND_SLASH=True导致POST请求重定向失败
- **解决**：在post_urls.py中添加文件上传路由，同时支持upload和upload/URL

技术要点

1. 后端技术

- Django模型扩展：为Message模型添加image和video字段
- 序列化器定制：实现自定义的validate和create方法处理媒体文件上传
- API设计：设计支持FormData文件上传的API端点
- 路由配置：确保支持带斜杠和不带斜杠的URL
- 媒体文件处理：配置媒体文件存储和URL生成

2. 前端技术

- React Hooks：使用useState、useEffect管理组件状态
- Ant Design组件：使用Upload、Image、Video等组件处理媒体文件
- FormData处理：实现手动文件上传，支持图片和视频
- 条件渲染：根据消息发送者动态渲染消息位置和样式
- API调用：支持普通JSON请求和FormData文件上传

3. 前后端协同

- 媒体文件URL生成：确保前端生成正确的媒体文件访问URL
- 文件上传处理：实现前端文件选择、预览和后端处理的完整流程
- 消息数据处理：确保消息数据正确传递和显示